

Brady Thibaudeau

Professor Cates

Intro to Data Science

11 May 2026

NBA All-Star Classification

Project Write-Up

1. Project Summary

For this project, I built a binary classifier to predict whether a given NBA player season qualifies as an All-Star selection. I collected season average statistics for every player across 27 NBA seasons (1999-2000 through 2025-26) using the NBA Stats API, which gave me a dataset of 13,246 player season rows. I then trained three classification models on seasons prior to 2018-19 and tested them on the seven most recent seasons. Logistic Regression came out on top in cross validation with a ROC-AUC of 0.988 and held up well on the test set at 0.976. The results show that box score stats are genuinely useful for identifying All-Star caliber seasons after the fact, though the roughly 5% positive class rate and the non statistical nature of the actual selection process make it a trickier problem than the accuracy numbers alone suggest.

2. Process and Methods

Data Collection

I pulled player statistics from the NBA Stats API using the `nba_api` Python library, specifically the `leaguedashplayerstats` endpoint. The script I wrote (`nba_season_averages.py`) looped over every season from 1999-2000 through 2025-26 and fetched per-game averages for the Regular Season. For each season I made a second API call using the All-Star season type to get the list of All-Star participants, then used that to create a binary `ALL_STAR` label on each player's regular season row. I added rate limiting pauses between seasons and exponential back off on failures to keep the API from timing out. Everything got saved to a single combined CSV at the end.

Preprocessing and Feature Engineering

I dropped identifier and non informative columns like `PLAYER_ID`, `PLAYER_NAME`, `TEAM_ABBREVIATION`, `SEASON`, and all the rank columns, which left me with 27 numeric features covering things like games played, win and loss record, minutes, shooting percentages, rebounds, assists, steals, blocks, turnovers, fouls, points, and plus/minus. I used median imputation and standard scaling inside a `sklearn Pipeline` so nothing would leak between train and test.

Train/Test Split

Rather than a random split, I used a temporal one: everything before the 2018-19 season went into training (8,783 rows) and everything from 2018-19 onward became the holdout test set

(4,463 rows). This way the model never sees future seasons during training, which gives a much more honest picture of how it would actually perform going forward.

Model Selection

I ran 5 fold stratified cross validation on the training set for all three models: Logistic Regression, Decision Tree, and KNN. For Logistic Regression and the Decision Tree I used `class_weight='balanced'` to account for the class imbalance, and for KNN I used `k=15` with distance weighting. I evaluated each model on ROC-AUC, Average Precision, and F1, then retrained the best one on the full training set before evaluating on the holdout. I also ran all three models on the holdout side by side so I could compare their precision and recall tradeoffs directly.

3. Results

Here is how each model performed during 5 fold cross validation on the training set:

Model	CV ROC-AUC	CV Avg Precision	CV F1
Logistic Regression	0.9881	0.8181	0.6378
KNN	0.9757	0.7863	0.6462
Decision Tree	0.9176	0.6123	0.6143

Logistic Regression had the best cross validated ROC-AUC, so it was selected as the primary model. Here is how all three compared on the holdout test set:

Model	ROC-AUC	Avg Precision	Precision	Recall	F1
KNN	0.959	0.677	0.684	0.677	0.680
Logistic Regression	0.976	0.720	0.403	0.949	0.566
Decision Tree	0.924	0.593	0.400	0.879	0.550

Logistic Regression had the best holdout ROC-AUC (0.976) and Average Precision (0.720), but KNN actually produced the best class 1 F1 score (0.680) because it struck a better balance between precision and recall. Logistic Regression was tuned to catch as many All-Stars as possible, which pushed recall up to 0.949 but dropped precision to just 0.403. Per season F1 on the holdout ranged from 0.48 to 0.64, with a slight downward drift in the most recent seasons.

4. Discussion

The big takeaway is that box score statistics carry a lot of signal for identifying All-Star caliber seasons. A ROC-AUC above 0.97 means the model ranks players really well, which makes intuitive sense: players who average 25+ points with high assists on winning teams tend to

dominate All-Star ballots. The feature importance results backed this up, with team wins, field goal attempts, assists, and points as the strongest positive predictors, and personal fouls, losses, and win percentage working against a player's chances.

The false positive analysis was probably the most interesting part of the whole project. The model's most confident false positives were Joel Embiid, Giannis Antetokounmpo, Kevin Durant, and LeBron James, each flagged with over 99.99% confidence. These players were not actually selected as All-Stars in those specific seasons despite putting up elite numbers. Looking into it, the reasons included injuries that shortened their seasons, the fixed roster size, and the realities of coaches making selections. The model is not really wrong here in any meaningful sense as the stats genuinely look like All-Star seasons. The selection process just involves factors no amount of box score data can capture.

On the other end, the top false negatives included role players like Ryan Dunn and Trayce Jackson-Davis, who received Rising Stars All-Star selections in 2024-25 despite modest counting stats, and Dirk Nowitzki in his final season averaging just 7.3 points. That last one was clearly a sentimental pick, which is exactly the kind of thing the model was never going to predict from numbers alone.

One thing worth noting: this is a post hoc study. All the features are full season averages computed after the season ends, but All-Star selections actually happen mid season. So this is really measuring how well stats classify outcomes in retrospect, not how well you could predict them in real time.

5. Challenges and Problems Encountered

A few things made this project harder than expected:

- The NBA Stats API was unreliable. It is unofficial and rate limited, and a lot of requests timed out or threw errors partway through. I had to add exponential back off retry logic and sleep intervals between seasons to get through the full data pull without it failing.
- Class imbalance was a real issue. All-Stars are only about 5% of the dataset, so a model that just predicts nobody as an All-Star still gets 95% accuracy. I had to use `class_weight='balanced'` and lean on ROC-AUC and Average Precision instead of raw accuracy to actually measure whether the model was learning anything useful.
- I had to be careful about the train and test split. A random split would have let future seasons bleed into training, which would inflate the results. Using a strict temporal cutoff fixed this but required paying close attention to how the SEASON column was filtered.
- The false positives were confusing at first. Seeing so many high confidence errors initially looked like something had gone wrong. It took some digging to realize those were legitimately elite seasons from players who were not selected due to roster constraints or injuries, not actual model mistakes.

6. What I Learned

The biggest thing I took away from this project is that picking the right evaluation metric matters enormously. I kept looking at accuracy early on and it was almost meaningless given the class imbalance. Once I shifted focus to ROC-AUC and Average Precision, the results started making a lot more sense. Seeing the precision recall tradeoff play out concretely across three models, where logistic regression caught nearly every All-Star but also flagged tons of non-All-Stars while KNN was far more conservative and balanced, made the whole concept click in a way that

reading about it never quite did. I also came away with a much better appreciation for error analysis. Digging into the specific players the model got wrong was more informative than any of the aggregate numbers, and it changed how I thought about what the model was actually doing.